



June 11, 2025

The Best Python Dashboard Tools: Comparative Analysis With Practical Examples

A hands-on comparison of five top frameworks using real code and Plotly charts



By John Loewen, PhD

7 min read

[Download article](#) ▾

Contents 4



1). Now as with Streamlit, once we have the Shiny framework installed, we can run our application from a Terminal (or Command) prompt. I am running my application from within the Pycharm Terminal environment: You can see from the screenshot that the application loads into the default browser at the first available port at or above 8000. And the "shiny" result: Very similar to the Dash visualization we saw earlier. Recent statistics show that Shiny is growing in popularity — and is ideal for users familiar with reactive concepts.

- Voila

- Panel

- In Summary...

Data Visualization Techniques in Python

Working with global and regional datasets often means building maps and time-series charts with filters (e.g. country or region selectors).

But where the heck do you begin? Well, Python is a great starting point, but how do you know which libraries in Python will work best for the data that you are trying to present?

Let me show you around five Python dashboard frameworks — ***Streamlit, Dash, Shiny for Python, Voila, and Panel*** — with some practical examples focusing on ease of use, interactivity for data exploration, layout flexibility, typical use cases, and Plotly integration.

Streamlit

Streamlit is extremely easy to learn for Python users and it is well-suited for exploratory analysis (e.g. filtering GPI data by year or region via sliders and dropdowns).

Widgets (sliders, dropdowns) are generated with functions like `st.slider` without boilerplate code, and interactive charts (maps, line charts) display immediately with `st.plotly_chart`.

Its reactive loop updates charts instantly as you add code, making it great for quick prototyping.

Additionally, **Streamlit** syntax is very close to Python, and no HTML/CSS knowledge is needed.

For our examples, let's download a useful UN dataset called the Global Peace Index (GPI). The dataset can be downloaded from my Github [HERE](#) .

The original dataset looks like this:

Country	iso3c	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020
Afghanistan	AFG	3.095	3.191	3.040	3.021	3.154	3.258	3.212	3.316	3.362	3.412	3.513	3.520	3.592
Albania	ALB	1.806	1.779	1.770	1.797	1.799	1.816	1.810	1.806	1.761	1.774	1.714	1.712	1.771
Algeria	DZA	2.306	2.333	2.333	2.479	2.399	2.383	2.293	2.263	2.238	2.206	2.212	2.235	2.234
Angola	AGO	2.062	2.066	2.006	2.049	2.087	2.089	2.062	1.957	1.974	1.938	1.972	1.955	1.989
Argentina	ARG	1.818	1.946	1.947	1.909	1.872	2.001	1.905	1.984	1.981	1.936	1.914	1.978	1.948
Armenia	ARM	2.196	2.219	2.229	2.193	2.194	2.198	2.134	2.157	2.105	2.177	2.228	2.121	1.952
Australia	AUS	1.349	1.376	1.387	1.439	1.497	1.451	1.386	1.359	1.388	1.414	1.422	1.427	1.428
Austria	AUT	1.347	1.365	1.357	1.395	1.364	1.283	1.265	1.233	1.220	1.298	1.263	1.249	1.251
Azerbaijan	AZE	2.283	2.320	2.342	2.348	2.368	2.456	2.389	2.394	2.380	2.434	2.413	2.376	2.144
Bahrain	BHR	1.869	1.868	1.812	2.147	2.068	2.152	2.206	2.114	2.191	2.291	2.272	2.191	2.103
Bangladesh	BGD	2.039	2.028	2.110	2.134	2.158	2.094	2.012	1.998	2.006	1.969	2.022	2.109	2.091
Belarus	BLR	2.127	2.137	2.144	2.128	2.122	2.109	2.080	2.127	2.057	1.991	2.019	2.042	2.042

With this dataset, we can look at the overall global trend for peace by creating a line chart that shows the global average across the range of years in the dataset (2008–2022).

Now let's access the dataset using Python and display this chart using Streamlit!

Example Streamlit GPI Chart:

```

1 import streamlit as st
2 import pandas as pd
3 import plotly.express as px
4
5 df = pd.read_csv('global_peace_index.csv')
6 gpi_cols = [str(y) for y in range(2008, 2023)]
7 df_world = df[gpi_cols].mean().reset_index()
8 df_world.columns = ['Year', 'GPI']
9 df_world['Year'] = df_world['Year'].astype(int)

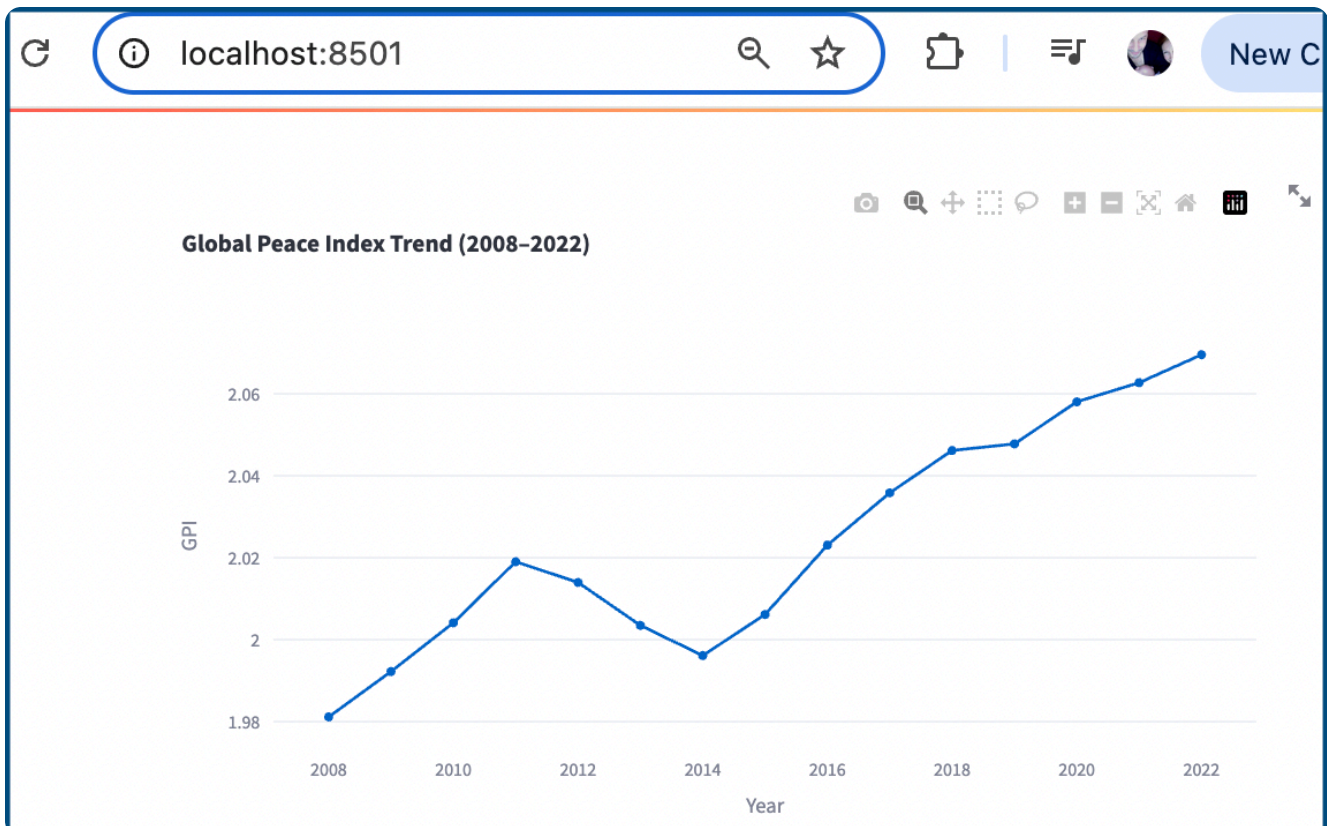
```

```
10 fig = px.line(df_world, x='Year', y='GPI', title='Global Peace Index Tren  
11 st.plotly_chart(fig, use_container_width=True)
```

To run this code, you need to have the streamlit framework installed. Then you can run it from a Terminal (command) prompt:

```
Terminal: Local x + v  
(venv) (base) loewenj700@Host-003 global_peace_index % streamlit run streamlit_comp01.py  
  
You can now view your Streamlit app in your browser.  
  
Local URL: http://localhost:8501  
Network URL: http://192.168.1.5:8501
```

The resulting Streamlit data visualization:



Perfect — super easy, super quick, super efficient code (less than 10 lines).

What we are looking at is the default layout with a single column (with an optional sidebar for widgets)

Streamlit recently added support for columns, tabs, and containers. These give some flexibility, but customization remains limited compared to other frameworks.

On the downside for Streamlit, inter-component reactivity is limited: only widgets (not charts) can serve as inputs to control code, and charts cannot natively send data to other components.

Streamlit has added chart event support (e.g. you can capture click or lasso selections via `st.plotly_chart(..., on_select="rerun")`), but this is less seamless than Dash's callbacks.

You cannot freely embed custom CSS or HTML — theming is handled by Streamlit's config or the new theming options. Customization can quickly become cumbersome and frustrating.

Dash (By Plotly)

Dash provides the most comprehensive Plotly integration and flexible control of layout and interactivity. This is because the Dash framework was written by the Plotly development team — go figure, right?

Dash allows every component (e.g. graphs, sliders, inputs) to be input/output in Python callbacks. This allows highly dynamic dashboards — like clicking a country in a choropleth map to update time series plots.

Here's the same line chart example created with Dash:

```
1 import dash
2 from dash import dcc, html
3 import pandas as pd
4 import plotly.express as px
5
6 app = dash.Dash(__name__)
7 df = pd.read_csv('global_peace_index.csv')
8 df_avg = df[[str(y) for y in range(2008, 2023)]].mean().reset_index()
9 df_avg.columns = ['Year', 'GPI']
10 df_avg['Year'] = df_avg['Year'].astype(int)
11 app.layout = html.Div([
12     dcc.Graph(
```

```
13     figure=px.line(df_avg, x='Year', y='GPI', title='Global GPI Trend')
14 )
15 ])
16 if __name__ == '__main__':
17     app.run_server(debug=True)
```

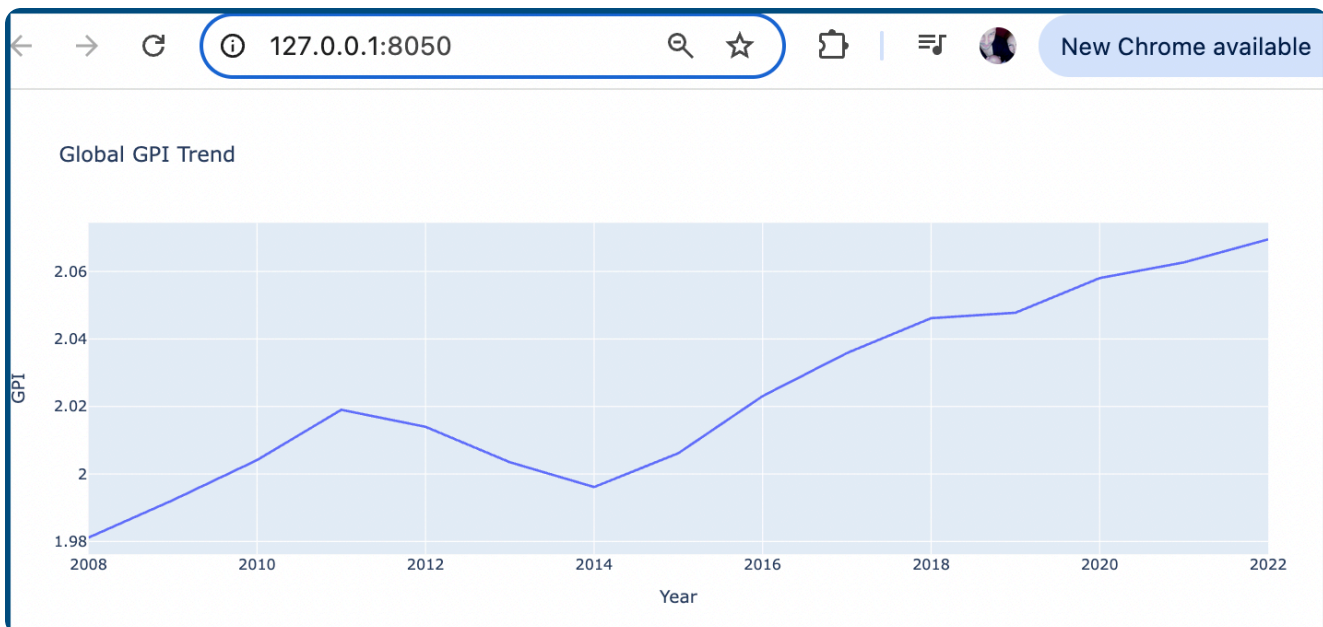
As you can see in the code, Dash apps use HTML-style components (`html.Div` , `dcc.Graph`) allowing full control. You can build responsive layouts and link UI components with CSS or Bootstrap.

We can run the Dash application from the terminal prompt, or also within our CGI editor (in my case, I am using PyCharm):

```
Run: dash_comp01 x
/Users/loewenj700/PycharmProjects/pythonExamplesApr19/venv/bin/python
Dash is running on http://127.0.0.1:8050/

* Serving Flask app 'dash_comp01'
* Debug mode: on
```

And the resulting chart is displayed in our default browser at the default port of 8050:



Awesome! We again have our chart displaying in our default browser.

Now as I mentioned above, the issue with Dash is that you need to build up the HTML using the built-in custom controls in Dash. This takes a bit more planning and work than the Streamlit framework, BUT it gives you more control of the layout and display.

This is why Dash is a good choice for production-grade dashboards.

Shiny for Python

Shiny offers reactive programming using decorators and reactive values. The reactive decorators in Shiny automatically update charts when input values change.

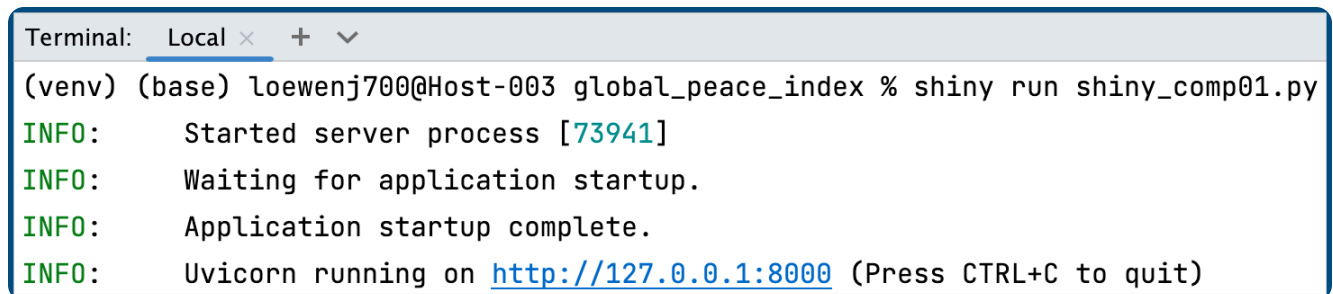
Our line chart example created with Shiny:

```
1  from shiny import App, ui, render
2  import pandas as pd
3  from shinywidgets import output_widget, render_widget
4  import plotly.express as px
5
6  # Load data
7  df = pd.read_csv("global_peace_index.csv")
8
9  # Compute average GPI per year
10 df_avg = df[[str(y) for y in range(2008, 2023)]].mean().reset_index()
11 df_avg.columns = ['Year', 'GPI']
12 df_avg['Year'] = df_avg['Year'].astype(int)
13
14 # Define UI
15 app_ui = ui.page_fluid(
16     ui.h2("GPI Time Series"),
17     output_widget("gpi_plot") # for Plotly chart
18 )
19
20 # Define server logic
21 def server(input, output, session):
22     @output()
23     @render_widget
24     def gpi_plot():
25         fig = px.line(df_avg, x='Year', y='GPI', title='GPI Trend (2008-2
26         return fig
27
```

```
28 # App
29 app = App(app_ui, server)
```

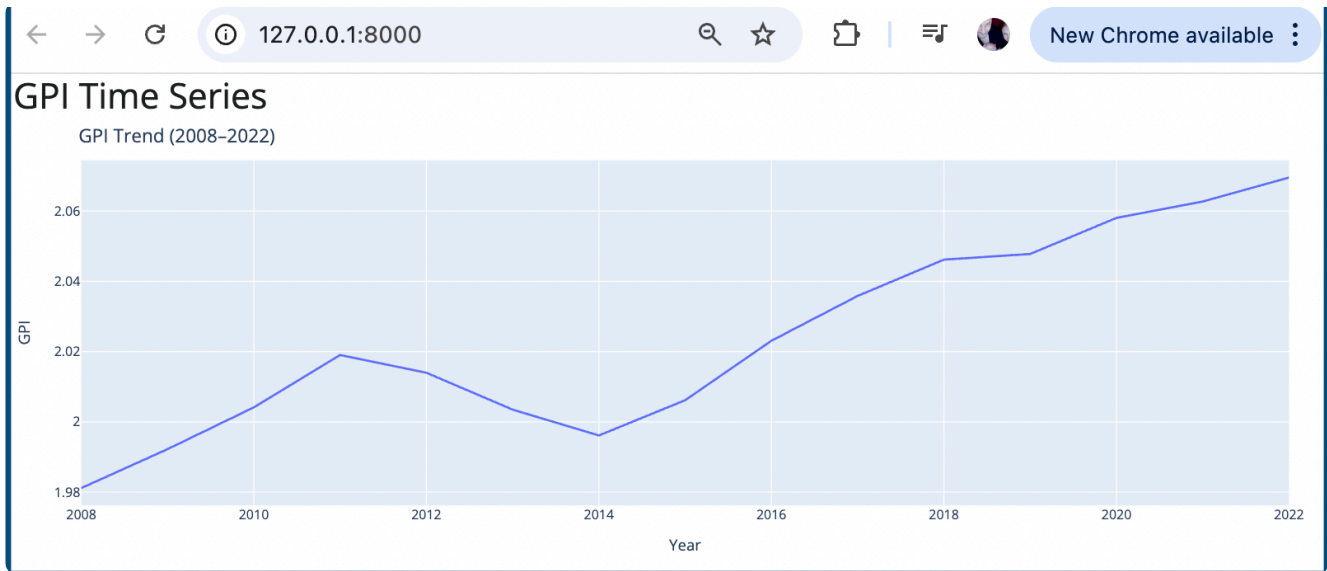
Shiny provides fluid layouts and theming through helper functions and using the **shinywidgets** library. As with the Dash framework, a cursory knowledge of HTML is needed in order to create the layout that supports your dashboard (ie. Heading tags like).

Now as with **Streamlit**, once we have the Shiny framework installed, we can run our application from a Terminal (or Command) prompt. I am running my application from within the Pycharm Terminal environment:



```
Terminal: Local x + v
(venv) (base) loewenj700@Host-003 global_peace_index % shiny run shiny_comp01.py
INFO:      Started server process [73941]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

You can see from the screenshot that the application loads into the default browser at the first available port at or above 8000. And the "shiny" result:



Very similar to the Dash visualization we saw earlier.

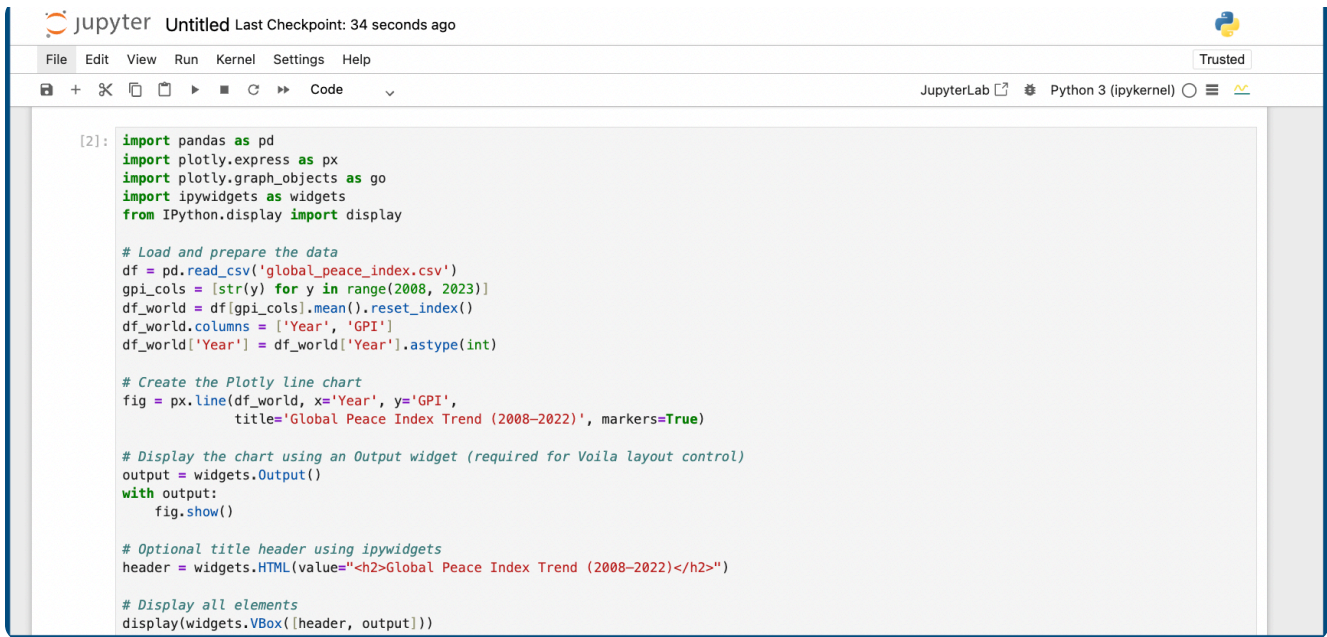
Recent statistics show that Shiny is growing in popularity — and is ideal for users familiar with reactive concepts.

Voila

Voila is a dashboarding framework that converts **Jupyter notebooks** directly into interactive web applications — very cool.

Unlike Streamlit, Dash, Panel, or Shiny, which require writing dedicated scripts or applications, Voila uses existing `.ipynb` notebooks as the app source. This makes it perfect for data scientists who are already comfortable exploring and building in Jupyter.

Here is the code for our line chart, loaded in Jupyter Notebook:



The screenshot shows a JupyterLab window titled 'Untitled' with a last checkpoint 34 seconds ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations, running, and code execution. The main area displays a Python notebook with the following code:

```
[2]: import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import ipywidgets as widgets
from IPython.display import display

# Load and prepare the data
df = pd.read_csv('global_peace_index.csv')
gpi_cols = [str(y) for y in range(2008, 2023)]
df_world = df[gpi_cols].mean().reset_index()
df_world.columns = ['Year', 'GPI']
df_world['Year'] = df_world['Year'].astype(int)

# Create the Plotly line chart
fig = px.line(df_world, x='Year', y='GPI',
              title='Global Peace Index Trend (2008-2022)', markers=True)

# Display the chart using an Output widget (required for Voila layout control)
output = widgets.Output()
with output:
    fig.show()

# Optional title header using ipywidgets
header = widgets.HTML(value="<h2>Global Peace Index Trend (2008-2022)</h2>")

# Display all elements
display(widgets.VBox([header, output]))
```

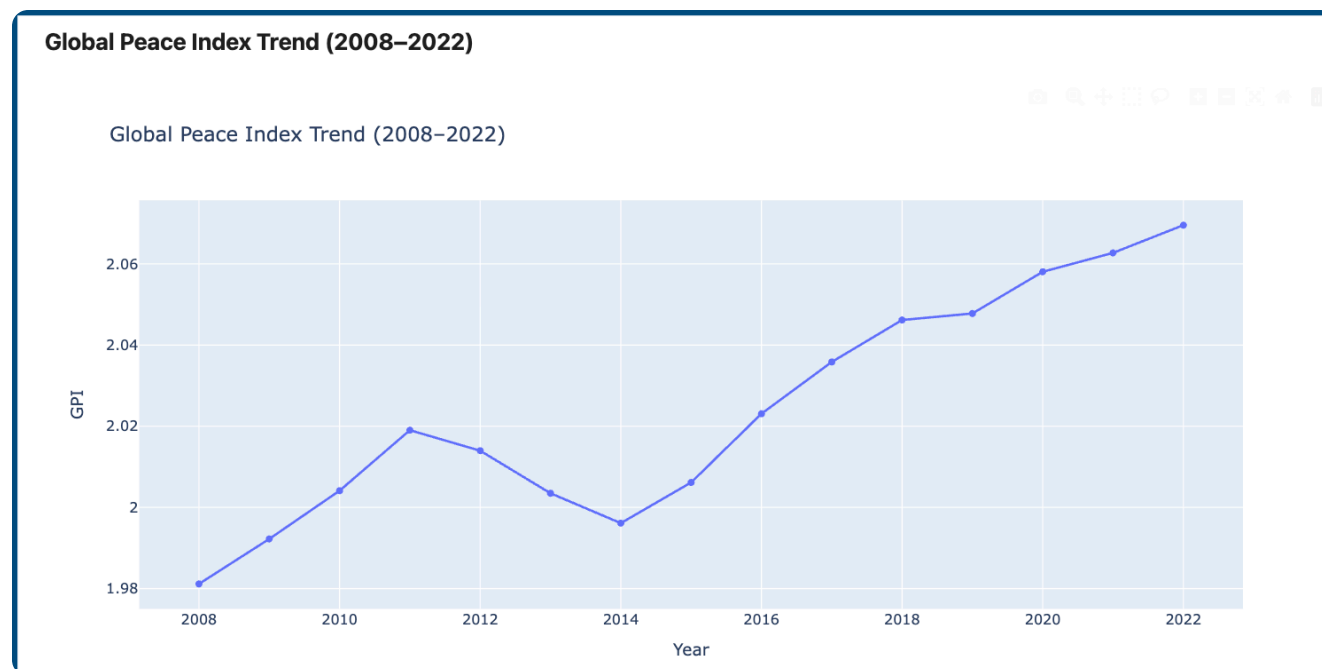
The actual code:

```
1 import pandas as pd
2 import plotly.express as px
3 import plotly.graph_objects as go
4 import ipywidgets as widgets
5 from IPython.display import display
6
7 # Load and prepare the data
8 df = pd.read_csv('global_peace_index.csv')
9 gpi_cols = [str(y) for y in range(2008, 2023)]
10 df_world = df[gpi_cols].mean().reset_index()
11 df_world.columns = ['Year', 'GPI']
12 df_world['Year'] = df_world['Year'].astype(int)
13
14 # Create the Plotly line chart
15 fig = px.line(df_world, x='Year', y='GPI',
16              title='Global Peace Index Trend (2008-2022)', markers=True)
17
18 # Display the chart using an Output widget (required for Voila layout control)
19 output = widgets.Output()
20 with output:
21     fig.show()
22
23 # Optional title header using ipywidgets
24 header = widgets.HTML(value="<h2>Global Peace Index Trend (2008-2022)</h2>")
```

```
25
26 # Display all elements
27 display(widgets.VBox([header, output]))
```

Voila follows the notebook layout, and it can be styled with templates.

The resulting data visualization (displayed within our Jupyter Notebook environment):



Great! It loads and displays this chart seamlessly as well.

As mentioned above, if you are familiar with Jupyter notebooks (as I am), then **Voila** is terrific for sharing existing notebooks as apps — you can even do this through GitHub and myBinder.org.

I wrote an article about this a few months ago (that was Boosted, btw):

[How Python Voila Can Be Your New Killer Visualization Superpower](#) [↗](#) *Creating public dashboards with Viola and Jupyter Notebooks*

If you want more detailed information on how to publicly implement a dashboard using Viola, give it a read!

Panel

Our last example here, **Panel**, provides powerful and flexible dashboarding. It is particularly good for combining multiple interactive widgets with reactive plots, and supports varying layout structures.

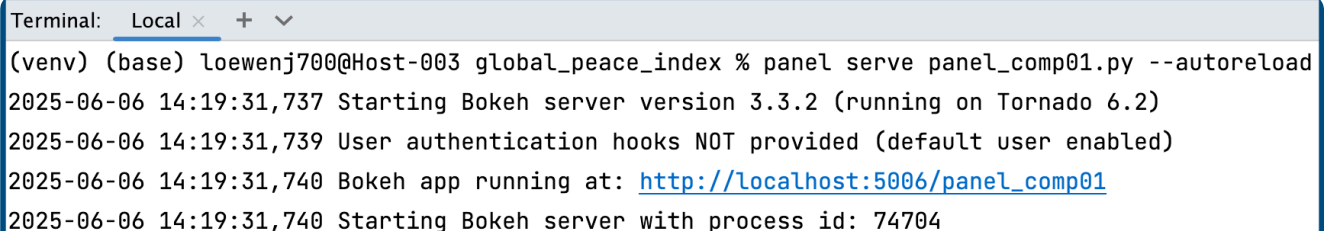
Our line chart example created with Panel:

```
1 import panel as pn
2 import pandas as pd
3 import plotly.express as px
4 pn.extension('plotly')
5
6 df = pd.read_csv('global_peace_index.csv')
7 df_avg = df[[str(y) for y in range(2008, 2023)]].mean().reset_index()
8 df_avg.columns = ['Year', 'GPI']
9 df_avg['Year'] = df_avg['Year'].astype(int)
10 plot = px.line(df_avg, x='Year', y='GPI', title='GPI Trend')
11 pn.pane.Plotly(plot).servable()
```

With Panel, widgets like `pn.widgets.Select` can bind to Plotly charts using `pn.bind` or reactive Param classes.

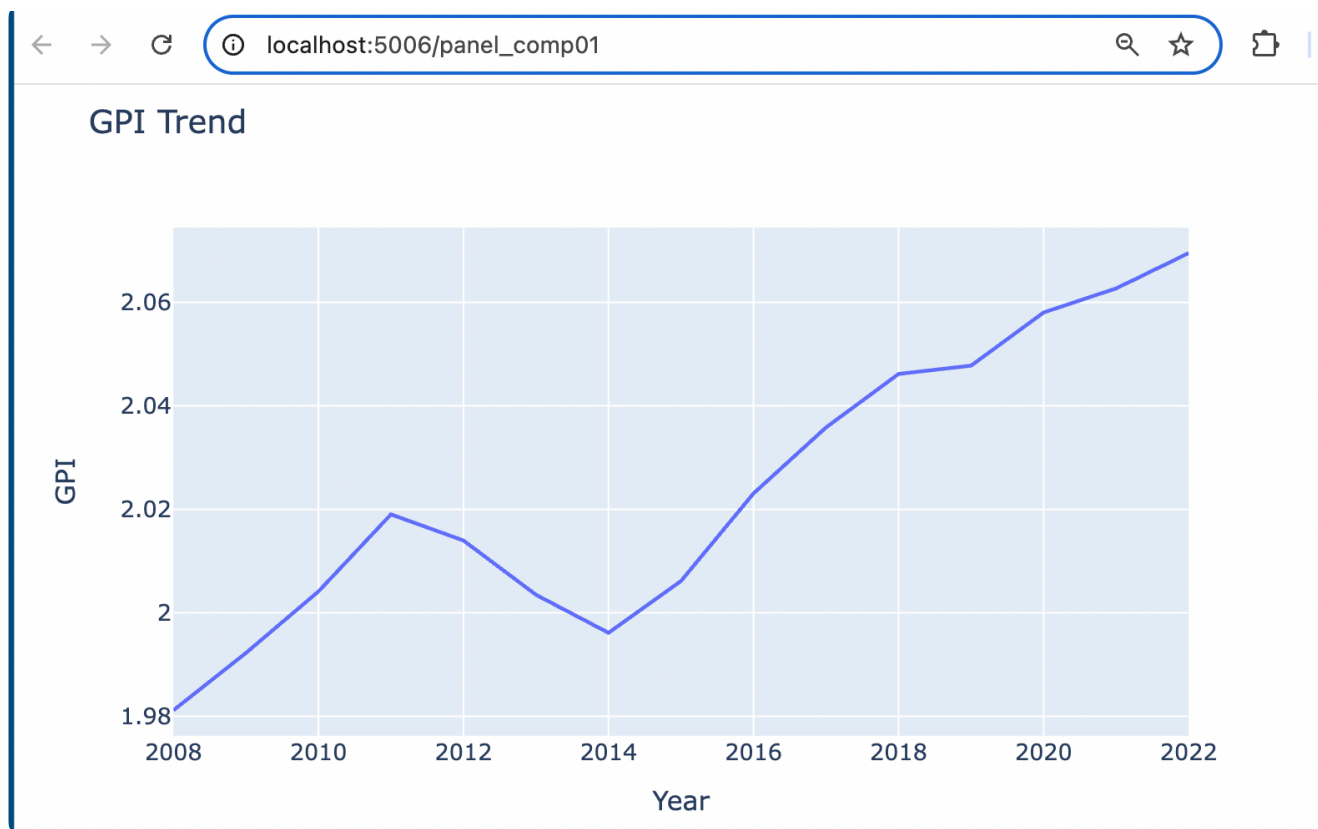
Panel also supports tabs, templates, multi-page views, and responsive design.

As with the Streamlit and Shiny framework, we can run our Panel application from the Terminal (or Command) prompt. I ran my code from the built-in PyCharm Terminal prompt:



```
Terminal: Local x + v
(venv) (base) loewenj700@Host-003 global_peace_index % panel serve panel_comp01.py --autoreload
2025-06-06 14:19:31,737 Starting Bokeh server version 3.3.2 (running on Tornado 6.2)
2025-06-06 14:19:31,739 User authentication hooks NOT provided (default user enabled)
2025-06-06 14:19:31,740 Bokeh app running at: http://localhost:5006/panel_comp01
2025-06-06 14:19:31,740 Starting Bokeh server with process id: 74704
```

And the resulting Panel application is displayed in the default browser at port 5006:



Terrific, super easy.

And folks, that's 5 common Python Dashboard frameworks and libraries in a nutshell. All 5 are able to seamlessly integrate these charts using Python Plotly — which is great as Plotly gives us additional interactivity options (above what traditional libraries like matplotlib can do).

Give each one of them a go to see what they can do!

In Summary...

To give you a bit more direction on where you would best use each framework:

- **Streamlit** is best for fast internal dashboards with minimal setup. Great for beginners and exploratory apps.
- **Dash** offers a high level of interactivity (through callback functions) and layout flexibility. Ideal for polished, multi-page dashboards.
- **Shiny** gives you reactive logic in a clean decorator style. Useful if you prefer a declarative model.

- **Voila** works best when working in Jupyter Notebooks and you need a quick, shareable dashboard. Can be used with GitHub and myBinder.org to provide publicly accessible dashboards.
- **Panel** bridges notebook and script workflows with a Pythonic reactive model. Good for scalable exploratory tools (perhaps better than Streamlit).

Each framework integrates well with Plotly — from simple rendering to full interactivity.

If this type of story is right up your alley, and you want to support me as a writer, subscribe to my [Substack](#) ↗.

[Subscribe to Data at Depth](#) ↗ *Welcome to Data at Depth! Our site offers articles on data analysis and visualization with Python/AI. Learn to create...*

On Substack, I publish a newsletter and articles that you won't find on the other platforms where I create content.

